CS4243 Sem2 25/26 **Lab 3: Image Segmentation** **Deadline: April 1st, 22:00**

Instruction

This lab has two parts. For each part, implement the outlined functions in `lab3.py` and run them in the notebook `lab3.ipynb`. For open questions, write your answers directly into the notebook. Expected outputs of qualitative results are supplied as reference, but your outputs do not need to match exactly.

Upload to Canvas your completed `lab3.py` and `lab3.ipynb` by zipping them into a file named `AXXX1_AXXX2.zip`, where `AXXX` is the student number of the group members. Submit one file per group. Missing files, incorrectly formatted code that does not run, etc. will be penalized.

Before submitting, please run **Kernel → Restart & Run All** in Jupyter to ensure your notebook executes from top to bottom without errors. The TA will grade primarily based on the cell outputs visible in your submitted `lab3.ipynb`.

He Qiyuan is the TA in charge of this lab. Please post any questions onto the Canvas forum or attend one of the FAQ sessions during the Lab session.

Objective

In Lecture 5, we learned about image segmentation as a clustering problem, including K-means and Mean-Shift clustering. In Lecture 6, we learned how to describe textures using filter banks (Gabor filters) and textons. In this lab, we will implement these concepts for image segmentation.

In the first part, you will implement the Mean-Shift segmentation algorithm and compare it with K-means. In the second part, you will build a Gabor-based filter bank, learn a texton dictionary, and use texture features for segmentation.

Part 1: Mean-Shift Segmentation (45%)

To do segmentation, an initial idea is to use K-means clustering on RGB values directly. We have provided the K-means segmentation code for your reference. In this part, you need to implement Mean-Shift clustering and compare it with K-means algorithm.

Feature Space Construction (5%)

- `construct_feature_space()` converts the input RGB image into a feature space for Mean-Shift segmentation.
  1. For each pixel at position (row, col) with colour (R, G, B), create a 5D feature: (spatial_weight × row, spatial_weight × col, R, G, B).
  2. The spatial_weight parameter controls the relative importance of spatial proximity versus colour similarity (cf. L05 slide 20).
  3. Return the feature array of shape (H×W, 5) with dtype float64.

Mean-Shift Update Step (15%)

- `mean_shift_step()` computes a single Mean-Shift update for a given point using a Gaussian kernel.
  1. Compute the Euclidean distance from point to every data point in data.
  2. Apply a Gaussian kernel: $w_i = \exp(-\|x_i - \text{point}\|^2 / (2 \cdot \text{bandwidth}^2))$.
  3. Compute the weighted mean: $\text{new_point} = \Sigma(w_i \cdot x_i) / \Sigma(w_i)$.
  4. Return the updated point.

Mean-Shift Segmentation (15%)

- `mean_shift_segmentation()` runs the full Mean-Shift algorithm to segment the image.
  1. For each data point, iteratively apply mean_shift_step() until the shift distance falls below threshold or max_iter is reached. Record the converged position (the mode).

2. Merge nearby modes: group modes that are within bandwidth / 2 of each other into the same cluster.
3. Assign each data point a segment label based on its merged mode.
4. Return the label array of shape (N,).

Implementation tip: Mean-Shift is computationally expensive. For faster testing, consider downscaling images. You may also refer to the speedup strategies on L05 slides 38–39.

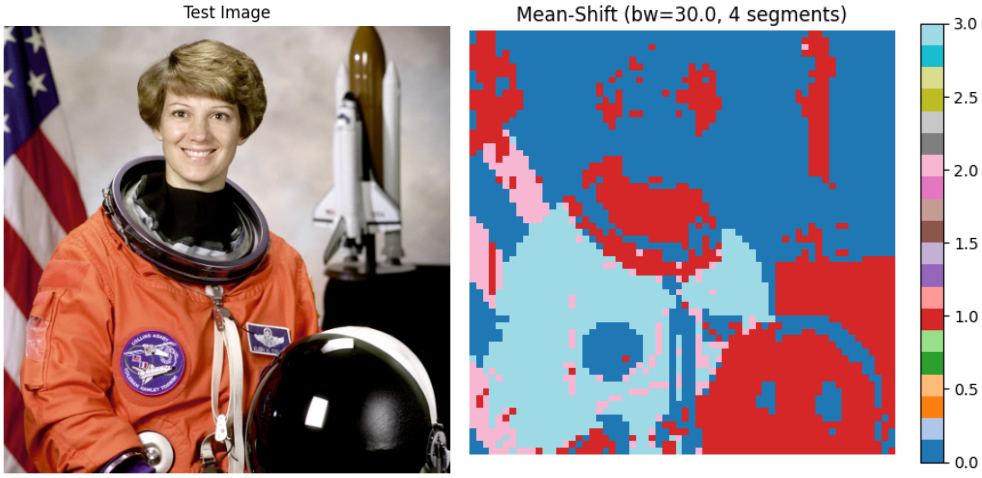

Fig.1 Expected Mean-Shift segmentation result.

Bandwidth Experiment and Comparison (5%)

Run Mean-Shift with different bandwidth values (e.g. 15, 30, 60) and observe the effect on the number and shape of segments. Visualize results side-by-side and compare with K-means.

Exploratory Questions (5%)

- How does the bandwidth parameter affect the Mean-Shift segmentation result? Discuss its relationship to the number of segments discovered.
- Compare K-means and Mean-Shift segmentation results on the same image. What are the advantages and disadvantages of each method?

<u>Part 2: Texture Description and Segmentation (55%)</u>

In Lecture 6, we learned that textures can be represented by their responses to a bank of filters. In this part, you will build a filter bank using Gabor filters, learn a texton dictionary, and use texture features for image segmentation.

Gabor Filter Construction (10%)

A Gabor filter combines a Gaussian envelope with a sinusoidal wave. It is sensitive to texture at a specific orientation and scale, making it well-suited for filter banks (cf. L06 slides 15–19).

- `create_gabor_filter()` constructs a 2D Gabor filter kernel. The Gabor function is defined as:

$$g(x, y) = \exp(-(x'^2 + \gamma^2 y'^2)/(2\sigma^2)) \cdot \cos(2\pi \cdot x'/\lambda + \psi)$$

where $x' = x \cdot \cos(\theta) + y \cdot \sin(\theta)$, $y' = -x \cdot \sin(\theta) + y \cdot \cos(\theta)$, and $\gamma = 0.5$ (fixed aspect ratio). Kernel size: $(6\sigma + 1) \times (6\sigma + 1)$.

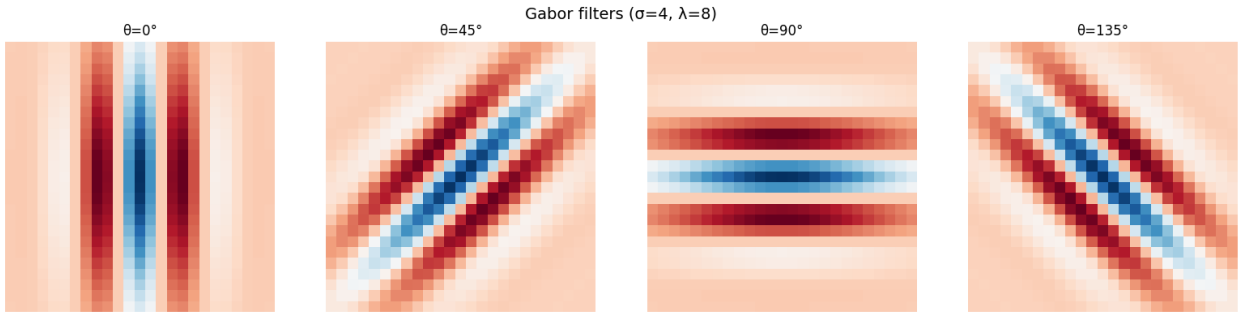

Fig.2 Expected Results of Gabor Filters Visualization.

Laplacian of Gaussian Filter (5%)

- `create_log_filter()` returns a Laplacian of Gaussian (LoG) kernel. The LoG filter detects "spot"-like patterns at a given scale. Kernel size: $(6\sigma+1) \times (6\sigma+1)$.

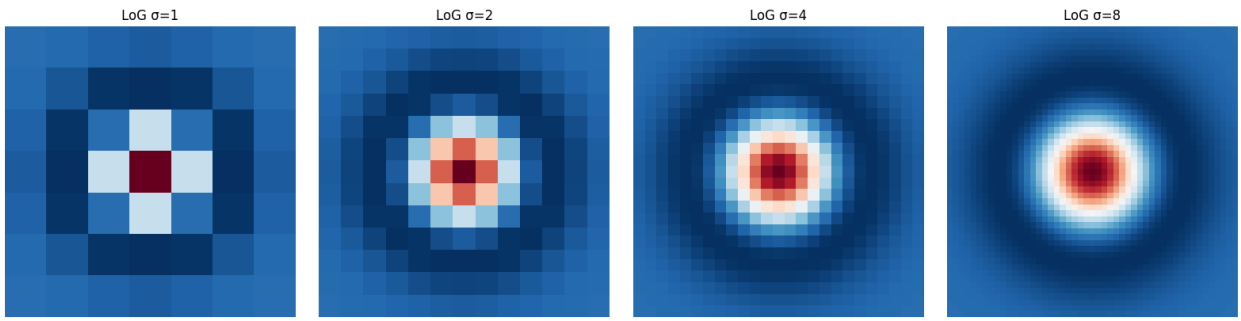

Fig.3 Expected Results of LoG Filters Visualization.

Filter Bank Response Extraction (10%)

The image is convolved with a 12-dimensional filter bank. The filter bank consists of Gabor filters at 4 orientations (0°, 45°, 90°, 135°) × 2 scales ($\sigma=2,\lambda=4$ and $\sigma=4,\lambda=8$), producing 8 filter responses, and LoG at 4 scales ($\sigma=1, 2, 4, 8$), producing 4 filter responses. All filters are applied to the grayscale image.

- `build_filter_bank_responses()` returns 12-dimensional feature vectors for the input image.
  1. Convert the image to grayscale.
  2. Apply all 12 filters (8 Gabor + 4 LoG) to the grayscale image.
  3. Return per-pixel feature vectors of shape (H, W, 12).

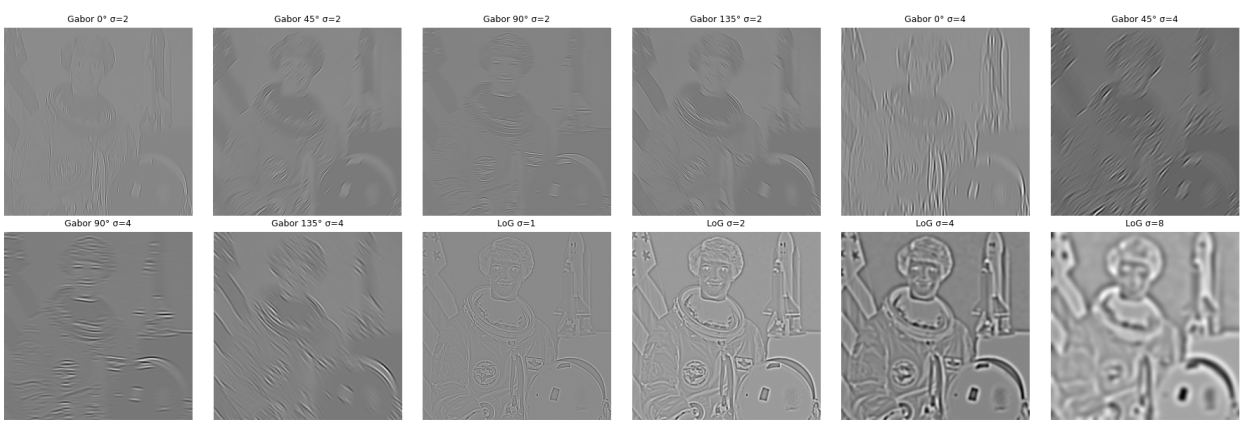

Fig.4 Expected Results of Filter Bank Response Extraction.

Texton Dictionary Learning (10%)

Next, you will learn textons by clustering the filter bank responses from training images (the MSRC dataset) using K-means. Each texton is represented by a cluster center.
Implement the `TextonDictionary` class in `lab3.py`.

- `TextonDictionary.learn_dictionary()` (5%) takes all training images as input, extracts filter bank responses, clusters them using K-means, and stores the cluster centers. Hint: Use MiniBatchKMeans from sklearn.cluster.
- `TextonDictionary.assign_textons()` (5%) predicts the texton label for each pixel. For each pixel in the test image, it gets assigned to the nearest cluster center. The cluster ID is the texton ID. Hint: Use KDTree from sklearn.neighbors.

Fig.5 Expected Results of Original image vs Texton map.

Texton Histogram (10%)

You will then implement the texton histogram by computing the distribution of texton IDs within a local window, providing a more robust texture descriptor than raw texton IDs.

- `compute_texton_histogram()` gathers all texton IDs within a rectangular window around each pixel and computes a normalized histogram. The size of the window is controlled by the `window_size` parameter.
  1. For each pixel, gather all texton IDs within a `window_size` × `window_size` window.
  2. Compute the normalized histogram (summing to 1) over n_textons bins.
  3. Return the per-pixel histogram feature map of shape `(H, W, n_textons)`.
  4. Handle border pixels by only counting valid (in-bound) entries.

Texture-Based Segmentation (5%)

In the final step, compare two texture feature options for segmentation using K-means:

- (B) Spatially smoothed filter bank responses: apply Gaussian blur to each filter response channel, then run K-means on the smoothed features.
- (C) Texton histograms: run K-means on the per-pixel texton histogram features.

Apply both approaches and compare the results. Note: Texton histograms aggregate information over a window, which can blur segmentation boundaries. Spatially smoothed filter responses tend to preserve boundaries better.

Exploratory Questions (5%)

- How does the `window_size` parameter in the texton histogram affect the segmentation result? Analyze the trade-off between segmentation smoothness and boundary precision.
- Compare segmentation using spatially smoothed filter responses (option B) versus texton histograms (option C). Which gives better boundaries, and why?
- Examine the Gabor filter responses at different orientations (θ). How do they capture different texture patterns in the image? Give a concrete example.

Congratulations! You have reached the end of Lab 3.